User Manual - Memory

Generated at: 2026-04-30 04:08:38

Product Overview

End-user guide for analyzing a project folder and using generated reports and hosted docs.

Prerequisites and Setup

Install dependencies, prepare Python environment, and configure OPENAI_API_KEY for richer metadata.

Step-by-Step Usage

Run: python main.py --folder C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory
Open outputs/code_documentation.pdf
Optionally host docs with run_and_serve.py

Screenshots / UI Flow

Placeholder: add run-screen, output-PDF view, and docs navigation screenshots.

Sample Input and Output

Input folder: C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory
Outputs: code_documentation.pdf, function_descriptions.json, test_scenarios.json, user and implementation PDFs.

Common Errors and Troubleshooting

Fix invalid folder path, missing Python files, API key/rate limit issues, or strict docs build failures.

FAQ / Support

Route is project-relative, any folder can be analyzed, and debug reports can be used for support diagnosis.

Runtime Snapshot

- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\jobs\sleep_consolidation_job.py::SleepConsolidationJob.run_once
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\jobs\sleep_consolidation_job.py::SleepConsolidationJob.run_forever
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\workflow\orchestrator.py::WorkflowOrchestrator.run
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\storage\vector_store.py::VectorStore.search
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\memory\orchestration\attention_engine.py::AttentionEngine.analyze
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\memory\retrieval\context_tree.py::ContextTree.to_dict
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\memory\retrieval\persistent_tree_store.py::PersistentTreeStore.load
- C:\Users\jarvi\OneDrive\Desktop\Book

syntesizer\memory_input_clean\Memory\app\memory\retrieval\tree_retriever.py::TreeRetriever.retrieve
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\memory\routing\bert_router.py::predict_route
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\memory\routing\bert_router.py::warmup
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\chat.py::chat
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\debug.py::recent
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\debug.py::stats
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\debug.py::interactions
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\graph.py::graph_data
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\graph.py::graph_view
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\router_api.py::debug_route
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\router_api.py::router_stats
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\api\tree.py::get_tree
- C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory\app\mcp\server.py::mcp_search

1. step_1 - orchestrates workflow execution steps
2. step_2 - orchestrates workflow execution steps
3. step_3 - orchestrates workflow execution steps
4. step_4 - Handles search stage transition
5. step_5 - Handles cosine stage transition
6. step_6 - Handles analyze stage transition
7. step_7 - Handles to dict stage transition
8. step_8 - Handles walk stage transition
9. step_9 - loads and retrieves required data
10. step_10 - builds and composes output artifacts
11. step_11 - loads and retrieves required data
12. step_12 - Handles walk stage transition
13. step_13 - Predict intent scores using BERT semantic classification
14. step_14 - Pre-load the model into memory (call during startup if desired)
15. step_15 - Handles chat stage transition
16. step_16 - Handles recent stage transition
17. step_17 - Handles stats stage transition
18. step_18 - Handles interactions stage transition
19. step_19 - JSON graph data: center node + neighbors + labeled edges
20. step_20 - Interactive visual graph viewer using vis-network

Top Connected Functions (Current Run)

- `route` (query_router.py) -> connectivity 37
- `run` (orchestrator.py) -> connectivity 28
- `analyze_emotional_signals` (emotional_analyzer.py) -> connectivity 27

- `compact` (compaction_engine.py) -> connectivity 22
- `retrieve` (hybrid_retriever.py) -> connectivity 18
- `run_once` (sleep_consolidation_job.py) -> connectivity 17
- `extract` (memory_extractor.py) -> connectivity 17
- `analyze` (attention_engine.py) -> connectivity 16
- `store` (supermemory.py) -> connectivity 14
- `reflect` (reflect_engine.py) -> connectivity 14

Generated Outputs

Artifact	Path
Code Documentation PDF	outputs/code_documentation.pdf
Function Descriptions Asset	outputs/assets/function_descriptions.json
Test Scenarios Asset	outputs/test_scenarios.json
User Manual (Markdown)	outputs/user_manual.md
Project Guide (Markdown)	outputs/project_implementation_guide.md
User Manual (PDF)	outputs/user_manual.pdf
Project Guide (PDF)	outputs/project_implementation_guide.pdf